

```

import os
import math
import json
import random
from dotenv import load_dotenv
from huggingface_hub import login
import matplotlib.pyplot as plt
import numpy as np
import pickle
from collections import Counter

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler

from sklearn.feature_extraction.text import CountVectorizer
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess

from sklearn.svm import LinearSVR
from sklearn.ensemble import RandomForestRegressor

```

Hàm/Dòng lệnh	Chức năng
import os	Nhập module os để làm việc với các thao tác hệ điều hành như tệp và thư mục.
import math	Nhập module math để sử dụng các hàm toán học như căn bậc hai, logarithm, v.v.
import json	Nhập module json để xử lý dữ liệu định dạng JSON (chuyển đổi giữa Python và JSON).
import random	Nhập module random để thực hiện các thao tác ngẫu nhiên như sinh số ngẫu nhiên.
from dotenv import load_dotenv	Nhập hàm load_dotenv từ thư viện dotenv để tải các biến môi trường từ tệp .env.
from huggingface_hub import login	Nhập hàm login từ thư viện huggingface_hub để đăng nhập vào Hugging Face Hub.
import matplotlib.pyplot as plt	Nhập thư viện matplotlib.pyplot (với bí danh plt) để vẽ đồ họa và biểu đồ.
import numpy as np	Nhập thư viện numpy (với bí danh np) để làm việc với các mảng và tính toán số học.
import pickle	Nhập module pickle để lưu trữ và tải các đối tượng Python dưới dạng tệp nhị phân.
from collections import Counter	Nhập lớp Counter từ thư viện collections để đếm tần suất các phần tử trong một đối tượng.
import pandas as pd	Nhập thư viện pandas (với bí danh pd) để làm việc với dữ liệu dạng bảng (DataFrame).
import numpy as np	Nhập lại thư viện numpy (với bí danh np) lần nữa (có thể là một bản sao).
from sklearn.linear_model import LinearRegression	Nhập mô hình hồi quy tuyến tính (LinearRegression) từ thư viện sklearn.
from sklearn.metrics import mean_squared_error,	Nhập các hàm mean_squared_error và r2_score từ thư viện sklearn.metrics để tính toán các chỉ số đánh

r2_score	giá.
from sklearn.preprocessing import StandardScaler	Nhập StandardScaler từ sklearn.preprocessing để chuẩn hóa dữ liệu.
from sklearn.feature_extraction.te xt import CountVectorizer	Nhập CountVectorizer từ sklearn.feature_extraction.text để chuyển văn bản thành ma trận tần suất từ.
from gensim.models import Word2Vec	Nhập Word2Vec từ thư viện gensim để huấn luyện mô hình từ vừng nhúng (word embeddings).
from gensim.utils import simple_preprocess	Nhập hàm simple_preprocess từ gensim.utils để tiền xử lý văn bản (tokenization, lowercasing).
from sklearn.svm import LinearSVR	Nhập mô hình LinearSVR từ thư viện sklearn.svm để thực hiện hồi quy với phương pháp Support Vector Regression.

<pre> GREEN = "\033[92m" YELLOW = "\033[93m" RED = "\033[91m" RESET = "\033[0m" COLOR_MAP = {"red":RED, "orange": YELLOW, "green": GREEN} load_dotenv(override=True) os.environ['OPENAI_API_KEY'] = os.getenv('OPENAI_API_KEY', 'your-key-if-not-using-env') os.environ['ANTHROPIC_API_KEY'] = os.getenv('ANTHROPIC_API_KEY', 'your-key-if-not-using-env') os.environ['HF_TOKEN'] = os.getenv('HF_TOKEN', 'your-key-if-not-using-env') hf_token = os.environ['HF_TOKEN'] login(hf_token, add_to_git_credential=True) from items import Item %matplotlib inline </pre>	
Hàm/Dòng lệnh	Chức năng
GREEN = "\033[92m"	Định nghĩa màu xanh lá cây cho đầu ra văn bản trong terminal (mã màu ANSI).
YELLOW = "\033[93m"	Định nghĩa màu vàng cho đầu ra văn bản trong terminal (mã màu ANSI).
RED = "\033[91m"	Định nghĩa màu đỏ cho đầu ra văn bản trong terminal (mã màu ANSI).
RESET = "\033[0m"	Định nghĩa mã màu để reset về màu mặc định trong terminal.
COLOR_MAP = {"red":RED, "orange": YELLOW, "green": GREEN}	Tạo một từ điển COLOR_MAP ánh xạ tên màu (red, orange, green) đến các mã màu ANSI tương ứng.
load_dotenv(override=True)	Tải các biến môi trường từ tệp .env và ghi đè nếu đã có giá trị trước đó.
os.environ['OPENAI_API_KEY'] = os.getenv('OPENAI_API_KEY', 'your-key-if-not-using-env')	Thiết lập biến môi trường OPENAI_API_KEY từ tệp .env hoặc giá trị mặc định.

EY', 'your-key-if-not-using-env')	
os.environ['ANTHROPIC_API_KEY'] = os.getenv('ANTHROPIC_API_KEY', 'your-key-if-not-using-env')	Thiết lập biến môi trường ANTHROPIC_API_KEY từ tệp .env hoặc giá trị mặc định.
os.environ['HF_TOKEN'] = os.getenv('HF_TOKEN', 'your-key-if-not-using-env')	Thiết lập biến môi trường HF_TOKEN từ tệp .env hoặc giá trị mặc định.
hf_token = os.environ['HF_TOKEN']	Lấy giá trị của HF_TOKEN từ biến môi trường và gán vào biến hf_token.
login(hf_token, add_to_git_credential=True)	Đăng nhập vào Hugging Face Hub bằng hf_token và thêm thông tin vào Git Credential.
from items import Item	Nhập lớp Item từ module items.
%matplotlib inline	Kích hoạt việc hiển thị trực tiếp biểu đồ trong notebook Jupyter (dành cho môi trường Jupyter).

<pre> with open('train.pkl', 'rb') as file: train = pickle.load(file) with open('test.pkl', 'rb') as file: test = pickle.load(file) print(train[0].prompt) print(train[0].price) </pre>	
Hàm/Dòng lệnh	Chức năng
with open('train.pkl', 'rb') as file:	Mở tệp train.pkl ở chế độ đọc nhị phân (rb) và gán đối tượng tệp vào biến file.
train = pickle.load(file)	Sử dụng pickle.load để giải nén và tải dữ liệu từ tệp train.pkl vào biến train.
with open('test.pkl', 'rb') as file:	Mở tệp test.pkl ở chế độ đọc nhị phân (rb) và gán đối tượng tệp vào biến file.
test = pickle.load(file)	Sử dụng pickle.load để giải nén và tải dữ liệu từ tệp test.pkl vào biến test.
print(train[0].prompt)	In giá trị prompt của phần tử đầu tiên trong danh sách train.
print(train[0].price)	In giá trị price của phần tử đầu tiên trong danh sách train.

```

class Tester:

    def __init__(self, predictor, title=None, data=test, size=250):
        self.predictor = predictor
        self.data = data
        self.title = title or predictor.__name__.replace("_", "
").title()
        self.size = size
        self.guesses = []
        self.truths = []
        self.errors = []
        self.sles = []
        self.colors = []

    def color_for(self, error, truth):
        if error<40 or error/truth < 0.2:
            return "green"
        elif error<80 or error/truth < 0.4:
            return "orange"
        else:
            return "red"

    def run_datapoint(self, i):
        datapoint = self.data[i]
        guess = self.predictor(datapoint)
        truth = datapoint.price
        error = abs(guess - truth)
        log_error = math.log(truth+1) - math.log(guess+1)
        sle = log_error ** 2
        color = self.color_for(error, truth)
        title = datapoint.title if len(datapoint.title) <= 40 else
datapoint.title[:40]+"..."
        self.guesses.append(guess)
        self.truths.append(truth)
        self.errors.append(error)
        self.sles.append(sle)
        self.colors.append(color)
        print(f"{COLOR_MAP[color]}{i+1}: Guess: ${guess:,.2f} Truth:
${truth:,.2f} Error: ${error:,.2f} SLE: {sle:,.2f} Item:
{title}{RESET}")

    def chart(self, title):
        max_error = max(self.errors)
        plt.figure(figsize=(12, 8))
        max_val = max(max(self.truths), max(self.guesses))
        plt.plot([0, max_val], [0, max_val], color='deepskyblue', lw=2,
alpha=0.6)
        plt.scatter(self.truths, self.guesses, s=3, c=self.colors)
        plt.xlabel('Ground Truth')
        plt.ylabel('Model Estimate')
        plt.xlim(0, max_val)
        plt.ylim(0, max_val)
        plt.title(title)
        plt.show()

```

```

def report(self):
    average_error = sum(self.errors) / self.size
    rmsle = math.sqrt(sum(self.sles) / self.size)
    hits = sum(1 for color in self.colors if color=="green")
    title = f"{self.title} Error=${average_error:,.2f}
    RMSLE={rmsle:,.2f} Hits={hits/self.size*100:.1f}%"
    self.chart(title)

def run(self):
    self.error = 0
    for i in range(self.size):
        self.run_datapoint(i)
    self.report()

@classmethod
def test(cls, function):
    cls(function).run()

def random_pricer(item):
    return random.randrange(1,1000)

```

Hàm/Dòng lệnh	Chức năng
class Tester:	Định nghĩa lớp Tester, dùng để kiểm thử mô hình dự đoán giá trị.
def __init__(self, predictor, title=None, data=test, size=250):	Khởi tạo đối tượng Tester, nhận các tham số: mô hình dự đoán predictor, tiêu đề title, dữ liệu data, và kích thước mẫu size.
self.predictor = predictor	Gán mô hình dự đoán vào thuộc tính predictor.
self.data = data	Gán dữ liệu vào thuộc tính data.
self.title = title or predictor.__name__.replace(" ", "").title()	Đặt tiêu đề cho đối tượng, nếu không có tiêu đề, sử dụng tên mô hình dự đoán.
self.size = size	Đặt kích thước mẫu (số lượng phần tử cần kiểm thử).
self.guesses = []	Khởi tạo danh sách trống để lưu các giá trị dự đoán.
self.truths = []	Khởi tạo danh sách trống để lưu các giá trị thực tế.
self.errors = []	Khởi tạo danh sách trống để lưu các lỗi (chênh lệch giữa dự đoán và giá trị thực tế).
self.sles = []	Khởi tạo danh sách trống để lưu các lỗi logarit bình phương (SLE).
self.colors = []	Khởi tạo danh sách trống để lưu màu sắc cho từng điểm dữ liệu dựa trên lỗi.
def color_for(self, error, truth):	Định nghĩa hàm color_for để gán màu sắc cho lỗi dựa trên độ chênh lệch giữa giá trị dự đoán và giá trị thực tế.
if error<40 or error/truth < 0.2:	Nếu lỗi nhỏ hơn 40 hoặc tỷ lệ lỗi trên giá trị thực tế nhỏ hơn 0.2, chọn màu xanh lá cây.
elif error<80 or error/truth < 0.4:	Nếu lỗi nhỏ hơn 80 hoặc tỷ lệ lỗi trên giá trị thực tế nhỏ hơn 0.4, chọn màu vàng.
else:	Nếu không thì chọn màu đỏ.
def run_datapoint(self, i):	Định nghĩa hàm run_datapoint để xử lý từng điểm dữ liệu trong tập kiểm thử.
datapoint = self.data[i]	Lấy điểm dữ liệu thứ i từ tập dữ liệu.

<code>guess = self.predictor(datapoint)</code>	Dự đoán giá trị cho điểm dữ liệu bằng mô hình predictor.
<code>truth = datapoint.price</code>	Lấy giá trị thực tế từ điểm dữ liệu.
<code>error = abs(guess - truth)</code>	Tính toán lỗi (chênh lệch tuyệt đối giữa giá trị dự đoán và giá trị thực tế).
<code>log_error = math.log(truth+1) - math.log(guess+1)</code>	Tính toán lỗi logarit (log_error).
<code>sle = log_error ** 2</code>	Tính lỗi logarit bình phương (SLE).
<code>color = self.color_for(error, truth)</code>	Gán màu sắc cho điểm dữ liệu dựa trên lỗi và giá trị thực tế.
<code>title = datapoint.title if len(datapoint.title) <= 40 else datapoint.title[:40]+"..."</code>	Cắt tiêu đề của điểm dữ liệu nếu quá dài (hơn 40 ký tự).
<code>self.guesses.append(guess)</code>	Thêm giá trị dự đoán vào danh sách guesses.
<code>self.truths.append(truth)</code>	Thêm giá trị thực tế vào danh sách truths.
<code>self.errors.append(error)</code>	Thêm lỗi vào danh sách errors.
<code>self.sles.append(sle)</code>	Thêm lỗi logarit bình phương vào danh sách sles.
<code>self.colors.append(color)</code>	Thêm màu sắc vào danh sách colors.
<code>print(f' {COLOR_MAP[color]} {i+1}: Guess: \${guess:,.2f} Truth: \${truth:,.2f} Error: \${error:,.2f} SLE: {sle:,.2f} Item: {title} {RESET}')</code>	In kết quả của điểm dữ liệu với màu sắc và thông tin dự đoán, thực tế, lỗi và tiêu đề.
<code>def chart(self, title):</code>	Định nghĩa hàm chart để vẽ biểu đồ dựa trên kết quả kiểm thử.
<code>max_error = max(self.errors)</code>	Lấy giá trị lỗi lớn nhất trong danh sách errors.
<code>plt.figure(figsize=(12, 8))</code>	Tạo một biểu đồ với kích thước 12x8 inch.
<code>max_val = max(max(self.truths), max(self.guesses))</code>	Lấy giá trị lớn nhất giữa các giá trị thực tế và dự đoán.
<code>plt.plot([0, max_val], [0, max_val], color='deepskyblue', lw=2, alpha=0.6)</code>	Vẽ một đường chéo từ (0, 0) đến (max_val, max_val) để so sánh giá trị thực tế và dự đoán.
<code>plt.scatter(self.truths, self.guesses, s=3, c=self.colors)</code>	Vẽ các điểm dữ liệu với màu sắc tương ứng.
<code>plt.xlabel('Ground Truth')</code>	Thiết lập nhãn trục x là "Ground Truth".
<code>plt.ylabel('Model Estimate')</code>	Thiết lập nhãn trục y là "Model Estimate".
<code>plt.xlim(0, max_val)</code>	Đặt giới hạn trục x từ 0 đến max_val.
<code>plt.ylim(0, max_val)</code>	Đặt giới hạn trục y từ 0 đến max_val.
<code>plt.title(title)</code>	Thiết lập tiêu đề cho biểu đồ.
<code>plt.show()</code>	Hiển thị biểu đồ.
<code>def report(self):</code>	Định nghĩa hàm report để báo cáo kết quả kiểm thử tổng quan.
<code>average_error =</code>	Tính toán lỗi trung bình.

<code>sum(self.errors) / self.size</code>	
<code>rmsle = math.sqrt(sum(self.sles) / self.size)</code>	Tính toán RMSLE (Root Mean Squared Logarithmic Error).
<code>hits = sum(1 for color in self.colors if color=="green")</code>	Tính số lượng điểm dữ liệu có màu xanh (mức độ chính xác cao).
<code>title = f'{self.title} Error=\${average_error:,.2f} RMSLE={rmsle:,.2f} Hits={hits/self.size*100:.1f} %'</code>	Tạo tiêu đề cho báo cáo với thông tin về lỗi, RMSLE và tỷ lệ hit.
<code>self.chart(title)</code>	Vẽ biểu đồ với tiêu đề báo cáo.
<code>def run(self):</code>	Định nghĩa hàm run để chạy kiểm thử trên toàn bộ dữ liệu.
<code>self.error = 0</code>	Khởi tạo biến error để lưu tổng lỗi.
<code>for i in range(self.size):</code>	Lặp qua từng điểm dữ liệu trong tập kiểm thử.
<code>self.run_datapoint(i)</code>	Gọi hàm run_datapoint để xử lý điểm dữ liệu thứ i.
<code>self.report()</code>	Gọi hàm report để báo cáo kết quả sau khi kiểm thử xong.
<code>@classmethod def test(cls, function):</code>	Định nghĩa phương thức lớp test để kiểm thử một hàm dự đoán cụ thể.
<code>cls(function).run()</code>	Tạo một đối tượng Tester và gọi phương thức run để thực hiện kiểm thử.
<code>def random_pricer(item):</code>	Định nghĩa một hàm ngẫu nhiên random_pricer để giả lập việc dự đoán giá trị cho một phần tử.
<code>return random.randrange(1,1000)</code>	Trả về một giá trị ngẫu nhiên trong khoảng từ 1 đến 1000.

```

random.seed(42)

Tester.test(random_pricer)

training_prices = [item.price for item in train]
training_average = sum(training_prices) / len(training_prices)

def constant_pricer(item):
    return training_average

Tester.test(constant_pricer)
train[0].details

for item in train:
    item.features = json.loads(item.details)
for item in test:
    item.features = json.loads(item.details)

train[0].features.keys()

feature_count = Counter()
for item in train:
    for f in item.features.keys():
        feature_count[f] += 1

feature_count.most_common(40)

```

Hàm/Dòng lệnh	Chức năng
random.seed(42)	Đặt seed cho quá trình tạo số ngẫu nhiên, đảm bảo kết quả lặp lại được.
Tester.test(random_pricer)	Kiểm thử mô hình random_pricer (dự đoán ngẫu nhiên) bằng lớp Tester.
training_prices = [item.price for item in train]	Lấy danh sách giá trị price từ tập huấn luyện train.
training_average = sum(training_prices) / len(training_prices)	Tính giá trị trung bình của các giá trị price trong tập huấn luyện.
def constant_pricer(item):	Định nghĩa hàm constant_pricer, trả về giá trị trung bình đã tính trước đó cho mỗi phần tử.
return training_average	Trả về giá trị trung bình từ biến training_average trong hàm constant_pricer.
Tester.test(constant_pricer)	Kiểm thử mô hình constant_pricer (dự đoán giá trị trung bình) bằng lớp Tester.
train[0].details	Lấy thuộc tính details của phần tử đầu tiên trong tập huấn luyện train.
for item in train:	Lặp qua tất cả phần tử trong tập huấn luyện train.
item.features = json.loads(item.details)	Chuyển chuỗi JSON trong details thành đối tượng Python (dictionaries) và gán vào features.
for item in test:	Lặp qua tất cả phần tử trong tập kiểm thử test.
item.features = json.loads(item.details)	Chuyển chuỗi JSON trong details thành đối tượng Python (dictionaries) và gán vào features.
train[0].features.keys()	Lấy tất cả các khóa (keys) trong dictionary features của phần tử đầu tiên trong tập huấn luyện.
feature_count = Counter()	Khởi tạo đối tượng Counter để đếm số lần xuất hiện

	của từng khóa trong features.
for item in train:	Lặp qua tất cả phần tử trong tập huấn luyện train.
for f in item.features.keys():	Lặp qua tất cả các khóa trong features của mỗi phần tử.
feature_count[f] += 1	Tăng số đếm của mỗi khóa trong feature_count.
feature_count.most_common(40)	Lấy 40 khóa xuất hiện nhiều nhất trong feature_count.

```
def get_weight(item):
    weight_str = item.features.get('Item Weight')
    if weight_str:
        parts = weight_str.split(' ')
        amount = float(parts[0])
        unit = parts[1].lower()
        if unit=="pounds":
            return amount
        elif unit=="ounces":
            return amount / 16
        elif unit=="grams":
            return amount / 453.592
        elif unit=="milligrams":
            return amount / 453592
        elif unit=="kilograms":
            return amount / 0.453592
        elif unit=="hundredths" and parts[2].lower()=="pounds":
            return amount / 100
        else:
            print(weight_str)
    return None
```

Hàm/Dòng lệnh	Chức năng
def get_weight(item):	Định nghĩa hàm get_weight, nhận đối tượng item để lấy thông tin trọng lượng.
weight_str = item.features.get('Item Weight')	Lấy giá trị của khóa Item Weight từ dictionary features trong đối tượng item.
if weight_str:	Kiểm tra xem giá trị weight_str có tồn tại hay không.
parts = weight_str.split(' ')	Chia chuỗi weight_str thành danh sách các phần tử (dùng dấu cách làm dấu phân cách).
amount = float(parts[0])	Lấy phần đầu tiên trong danh sách parts (số lượng trọng lượng) và chuyển thành kiểu float.
unit = parts[1].lower()	Lấy phần thứ hai trong danh sách parts (đơn vị trọng lượng) và chuyển thành chữ thường.
if unit == "pounds":	Kiểm tra nếu đơn vị là "pounds" (pound).
return amount	Trả về trọng lượng bằng đơn vị pound (không cần chuyển đổi).
elif unit == "ounces":	Kiểm tra nếu đơn vị là "ounces" (ounce).
return amount / 16	Chuyển đổi trọng lượng từ ounce sang pound (1 pound = 16 ounces).
elif unit == "grams":	Kiểm tra nếu đơn vị là "grams" (gram).
return amount / 453.592	Chuyển đổi trọng lượng từ gram sang pound (1 pound = 453.592 grams).
elif unit == "milligrams":	Kiểm tra nếu đơn vị là "milligrams" (miligram).

return amount / 453592	Chuyển đổi trọng lượng từ miligram sang pound (1 pound = 453592 milligrams).
elif unit == "kilograms":	Kiểm tra nếu đơn vị là "kilograms" (kilogram).
return amount / 0.453592	Chuyển đổi trọng lượng từ kilogram sang pound (1 pound = 0.453592 kilograms).
elif unit == "hundredths" and parts[2].lower() == "pounds":	Kiểm tra nếu đơn vị là "hundredths of pounds".
return amount / 100	Chuyển đổi trọng lượng từ "hundredths of pounds" sang pound (1 hundredth = 1/100 pound).
else:	Nếu đơn vị không hợp lệ.
print(weight_str)	In ra chuỗi trọng lượng nếu không nhận diện được đơn vị hợp lệ.
return None	Trả về None nếu không có trọng lượng hợp lệ hoặc không thể xác định đơn vị.

<pre> weights = [get_weight(t) for t in train] weights = [w for w in weights if w] average_weight = sum(weights)/len(weights) average_weight def get_weight_with_default(item): weight = get_weight(item) return weight or average_weight def get_rank(item): rank_dict = item.features.get("Best Sellers Rank") if rank_dict: ranks = rank_dict.values() return sum(ranks)/len(ranks) return None </pre>	
Hàm/Dòng lệnh	Chức năng
weights = [get_weight(t) for t in train]	Lấy trọng lượng từ mỗi phần tử trong tập huấn luyện train bằng hàm get_weight.
weights = [w for w in weights if w]	Loại bỏ các giá trị None trong danh sách weights (chỉ giữ lại các giá trị hợp lệ).
average_weight = sum(weights) / len(weights)	Tính giá trị trọng lượng trung bình từ danh sách weights.
average_weight	Trả về giá trị trọng lượng trung bình tính được từ weights.
def get_weight_with_default(item):	Định nghĩa hàm get_weight_with_default, nhận đối tượng item và trả về trọng lượng.
weight = get_weight(item)	Lấy trọng lượng từ đối tượng item bằng hàm get_weight.
return weight or average_weight	Nếu trọng lượng có giá trị (không phải None), trả về giá trị trọng lượng; ngược lại, trả về average_weight.
def get_rank(item):	Định nghĩa hàm get_rank, nhận đối tượng item và trả về giá trị xếp hạng trung bình.

<code>rank_dict = item.features.get("Best Sellers Rank")</code>	Lấy giá trị xếp hạng từ khóa Best Sellers Rank trong features của đối tượng item.
<code>if rank_dict:</code>	Kiểm tra nếu rank_dict có giá trị (không phải None).
<code>ranks = rank_dict.values()</code>	Lấy tất cả các giá trị xếp hạng từ rank_dict.
<code>return sum(ranks) / len(ranks)</code>	Tính và trả về giá trị trung bình của các xếp hạng.
<code>return None</code>	Trả về None nếu không tìm thấy xếp hạng trong rank_dict.

<pre> ranks = [get_rank(t) for t in train] ranks = [r for r in ranks if r] average_rank = sum(ranks)/len(ranks) average_rank def get_rank_with_default(item): rank = get_rank(item) return rank or average_rank def get_text_length(item): return len(item.test_prompt()) brands = Counter() for t in train: brand = t.features.get("Brand") if brand: brands[brand]+=1 brands.most_common(40) </pre>	
Hàm/Dòng lệnh	Chức năng
<code>ranks = [get_rank(t) for t in train]</code>	Lấy xếp hạng từ mỗi phần tử trong tập huấn luyện train bằng hàm get_rank.
<code>ranks = [r for r in ranks if r]</code>	Loại bỏ các giá trị None trong danh sách ranks (chỉ giữ lại các giá trị hợp lệ).
<code>average_rank = sum(ranks) / len(ranks)</code>	Tính giá trị xếp hạng trung bình từ danh sách ranks.
<code>average_rank</code>	Trả về giá trị xếp hạng trung bình tính được từ ranks.
<code>def get_rank_with_default(item):</code>	Định nghĩa hàm get_rank_with_default, nhận đối tượng item và trả về xếp hạng.
<code>rank = get_rank(item)</code>	Lấy xếp hạng từ đối tượng item bằng hàm get_rank.
<code>return rank or average_rank</code>	Nếu xếp hạng có giá trị (không phải None), trả về giá trị xếp hạng; ngược lại, trả về average_rank.
<code>def get_text_length(item):</code>	Định nghĩa hàm get_text_length, nhận đối tượng item và trả về độ dài văn bản trong test_prompt.
<code>return len(item.test_prompt())</code>	Trả về độ dài của văn bản trả về từ item.test_prompt().
<code>brands = Counter()</code>	Khởi tạo đối tượng Counter để đếm số lần xuất hiện của các nhãn hiệu trong brands.
<code>for t in train:</code>	Lặp qua tất cả phần tử trong tập huấn luyện train.
<code>brand =</code>	Lấy giá trị của khóa Brand từ features trong đối

t.features.get("Brand")	tượng t.
if brand:	Kiểm tra nếu có giá trị nhãn hiệu (brand).
brands[brand] += 1	Tăng số đếm của nhãn hiệu trong brands.
brands.most_common(40)	Lấy 40 nhãn hiệu xuất hiện nhiều nhất trong brands.

```

TOP_ELECTRONICS_BRANDS = ["hp", "dell", "lenovo", "samsung", "asus",
"sony", "canon", "apple", "intel"]
def is_top_electronics_brand(item):
    brand = item.features.get("Brand")
    return brand and brand.lower() in TOP_ELECTRONICS_BRANDS

def get_features(item):
    return {
        "weight": get_weight_with_default(item),
        "rank": get_rank_with_default(item),
        "text_length": get_text_length(item),
        "is_top_electronics_brand": 1 if is_top_electronics_brand(item)
    else 0
    }

get_features(train[0])

def list_to_dataframe(items):
    features = [get_features(item) for item in items]
    df = pd.DataFrame(features)
    df['price'] = [item.price for item in items]
    return df

train_df = list_to_dataframe(train)
test_df = list_to_dataframe(test[:250])

```

Hàm/Dòng lệnh	Chức năng
TOP_ELECTRONICS_BRANDS = ["hp", "dell", "lenovo", "samsung", "asus", "sony", "canon", "apple", "intel"]	Khai báo danh sách các thương hiệu điện tử hàng đầu.
def is_top_electronics_brand(item):	Khởi tạo hàm kiểm tra xem thương hiệu của sản phẩm có thuộc danh sách các thương hiệu điện tử hàng đầu hay không.
brand = item.features.get("Brand")	Lấy thông tin thương hiệu của sản phẩm từ thuộc tính "Brand".
return brand and brand.lower() in TOP_ELECTRONICS_BRANDS	Kiểm tra xem thương hiệu có tồn tại trong danh sách các thương hiệu hàng đầu, trả về True hoặc False.
def get_features(item):	Khởi tạo hàm lấy các thông tin tính năng của sản phẩm.
return {	Trả về một từ điển với các tính năng của sản phẩm.
"weight": get_weight_with_default(item),	Lấy thông tin trọng lượng của sản phẩm.

"rank": get_rank_with_default(item)	Lấy thông tin xếp hạng của sản phẩm.
"text_length": get_text_length(item),	Lấy độ dài của mô tả sản phẩm.
"is_top_electronics_brand": 1 if is_top_electronics_brand(ite m) else 0	Kiểm tra và đánh dấu sản phẩm thuộc thương hiệu hàng đầu, trả về 1 hoặc 0.
get_features(train[0])	Gọi hàm get_features để lấy thông tin tính năng của sản phẩm đầu tiên trong danh sách train.
def list_to_dataframe(items):	Khởi tạo hàm chuyển đổi danh sách sản phẩm thành dataframe.
features = [get_features(item) for item in items]	Duyệt qua từng sản phẩm trong danh sách items và lấy thông tin tính năng.
df = pd.DataFrame(features)	Chuyển danh sách các tính năng thành một dataframe.
df['price'] = [item.price for item in items]	Thêm cột giá vào dataframe từ thông tin giá của các sản phẩm.
return df	Trả về dataframe chứa thông tin tính năng và giá của các sản phẩm.
train_df = list_to_dataframe(train)	Gọi hàm list_to_dataframe để chuyển danh sách train thành dataframe train_df.
test_df = list_to_dataframe(test[:250])	Gọi hàm list_to_dataframe để chuyển 250 sản phẩm đầu tiên từ danh sách test thành dataframe test_df.

```

np.random.seed(42)

feature_columns = ['weight', 'rank', 'text_length',
'is_top_electronics_brand']

X_train = train_df[feature_columns]
y_train = train_df['price']
X_test = test_df[feature_columns]
y_test = test_df['price']

model = LinearRegression()
model.fit(X_train, y_train)

for feature, coef in zip(feature_columns, model.coef_):
    print(f"{feature}: {coef}")
print(f"Intercept: {model.intercept_}")

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"R-squared Score: {r2}")

```

Hàm/Dòng lệnh	Chức năng
<code>np.random.seed(42)</code>	Đặt giá trị hạt giống cho bộ sinh số ngẫu nhiên, giúp tái tạo kết quả mỗi khi chạy lại mã.
<code>feature_columns = ['weight', 'rank', 'text_length', 'is_top_electronics_brand']</code>	Định nghĩa danh sách các cột đặc trưng (features) sẽ được sử dụng trong mô hình học máy.
<code>X_train = train_df[feature_columns]</code>	Tạo tập dữ liệu huấn luyện cho đặc trưng (features) từ dataframe <code>train_df</code> .
<code>y_train = train_df['price']</code>	Tạo tập dữ liệu mục tiêu (target variable) từ cột "price" trong <code>train_df</code> .
<code>X_test = test_df[feature_columns]</code>	Tạo tập dữ liệu kiểm tra cho đặc trưng (features) từ dataframe <code>test_df</code> .
<code>y_test = test_df['price']</code>	Tạo tập dữ liệu mục tiêu từ cột "price" trong <code>test_df</code> .
<code>model = LinearRegression()</code>	Khởi tạo mô hình hồi quy tuyến tính.
<code>model.fit(X_train, y_train)</code>	Huấn luyện mô hình hồi quy tuyến tính trên dữ liệu huấn luyện <code>X_train</code> và mục tiêu <code>y_train</code> .
<code>for feature, coef in zip(feature_columns, model.coef_):</code>	Duyệt qua các đặc trưng và hệ số hồi quy tương ứng để in ra.
<code>print(f'{feature}: {coef}')</code>	In ra tên đặc trưng và hệ số hồi quy tương ứng.
<code>print(f'Intercept: {model.intercept_}')</code>	In ra hệ số chặn (intercept) của mô hình hồi quy tuyến tính.
<code>y_pred = model.predict(X_test)</code>	Dự đoán giá trị mục tiêu (price) cho tập kiểm tra <code>X_test</code> .
<code>mse = mean_squared_error(y_test, y_pred)</code>	Tính toán sai số bình phương trung bình (Mean Squared Error) giữa giá trị thực tế <code>y_test</code> và giá trị dự đoán <code>y_pred</code> .
<code>r2 = r2_score(y_test, y_pred)</code>	Tính toán điểm số R-squared (hệ số xác định) giữa giá trị thực tế <code>y_test</code> và giá trị dự đoán <code>y_pred</code> .
<code>print(f'Mean Squared Error: {mse}')</code>	In ra sai số bình phương trung bình.
<code>print(f'R-squared Score: {r2}')</code>	In ra điểm số R-squared của mô hình.

```
def linear_regression_pricer(item):
    features = get_features(item)
    features_df = pd.DataFrame([features])
    return model.predict(features_df)[0]
```

Test on test/linear_regression_pricer()

Hàm/Dòng lệnh	Chức năng
<code>def linear_regression_pricer(item):</code>	Khởi tạo hàm tính giá dựa trên mô hình hồi quy tuyến tính với đặc trưng được lấy từ sản phẩm.
<code>features = get_features(item)</code>	Lấy thông tin các đặc trưng của sản phẩm thông qua hàm <code>get_features</code> .
<code>features_df = pd.DataFrame([features])</code>	Chuyển đổi các đặc trưng của sản phẩm thành một dataframe.
<code>return</code>	Dự đoán giá trị sản phẩm từ mô hình hồi quy tuyến

<code>model.predict(features_df)[0]</code>	tính và trả về giá trị dự đoán đầu tiên.
<code>Tester.test(linear_regression_pricer)</code>	Chạy kiểm thử cho hàm <code>linear_regression_pricer</code> .
<code>prices = np.array([float(item.price) for item in train])</code>	Tạo mảng <code>prices</code> từ giá của các sản phẩm trong danh sách <code>train</code> .
<code>documents = [item.test_prompt() for item in train]</code>	Tạo danh sách các prompt của sản phẩm trong danh sách <code>train</code> .
<code>np.random.seed(42)</code>	Đặt giá trị hạt giống cho bộ sinh số ngẫu nhiên, giúp tái tạo kết quả mỗi khi chạy lại mã.
<code>vectorizer = CountVectorizer(max_features=1000, stop_words='english')</code>	Khởi tạo bộ vector hóa văn bản <code>CountVectorizer</code> để chuyển các prompt thành các đặc trưng số học, giới hạn tối đa 1000 đặc trưng và loại bỏ từ dừng.
<code>X = vectorizer.fit_transform(documents)</code>	Áp dụng <code>CountVectorizer</code> để chuyển danh sách <code>documents</code> thành ma trận đặc trưng.
<code>regressor = LinearRegression()</code>	Khởi tạo mô hình hồi quy tuyến tính.
<code>regressor.fit(X, prices)</code>	Huấn luyện mô hình hồi quy tuyến tính với các đặc trưng <code>X</code> và giá trị mục tiêu <code>prices</code> .

<pre>def bow_lr_pricer(item): x = vectorizer.transform([item.test_prompt()]) return max(regressor.predict(x)[0], 0) Tester.test(bow_lr_pricer) np.random.seed(42) processed_docs = [simple_preprocess(doc) for doc in documents] w2v_model = Word2Vec(sentences=processed_docs, vector_size=400, window=5, min count=1, workers=8)</pre>	
Hàm/Dòng lệnh	Chức năng
<code>def bow_lr_pricer(item):</code>	Khởi tạo hàm tính giá dựa trên mô hình hồi quy tuyến tính sử dụng phương pháp Bag-of-Words (BOW).
<code>x = vectorizer.transform([item.test_prompt()])</code>	Chuyển đổi prompt của sản phẩm thành dạng đặc trưng số học bằng cách sử dụng <code>vectorizer</code> đã được huấn luyện trước đó.
<code>return max(regressor.predict(x)[0], 0)</code>	Dự đoán giá trị cho sản phẩm từ mô hình hồi quy tuyến tính, đảm bảo giá trị dự đoán không nhỏ hơn 0.
<code>Tester.test(bow_lr_pricer)</code>	Chạy kiểm thử cho hàm <code>bow_lr_pricer</code> .
<code>np.random.seed(42)</code>	Đặt giá trị hạt giống cho bộ sinh số ngẫu nhiên, giúp tái tạo kết quả mỗi khi chạy lại mã.
<code>processed_docs</code>	Tiền xử lý các tài liệu văn bản trong <code>documents</code> bằng

[simple_preprocess(doc) for doc in documents]	cách sử dụng hàm simple_preprocess (tiền xử lý cơ bản như chuyển đổi về chữ thường, loại bỏ dấu câu, v.v.).
w2v_model = Word2Vec(sentences=processed_docs, vector_size=400, window=5, min_count=1, workers=8)	Huấn luyện mô hình Word2Vec với các tài liệu đã được tiền xử lý, sử dụng kích thước vector 400, cửa sổ 5 từ, và yêu cầu ít nhất 1 lần xuất hiện cho mỗi từ.

```
def document_vector(doc):
    doc_words = simple_preprocess(doc)
    word_vectors = [w2v_model.wv[word] for word in doc_words if word in w2v_model.wv]
    return np.mean(word_vectors, axis=0) if word_vectors else np.zeros(w2v_model.vector_size)

X_w2v = np.array([document_vector(doc) for doc in documents])

word2vec_lr_regressor = LinearRegression()
word2vec_lr_regressor.fit(X_w2v, prices)
```

Hàm/Dòng lệnh	Chức năng
def document_vector(doc):	Khởi tạo hàm tính vector đại diện cho một tài liệu (document) dựa trên các từ trong tài liệu đó bằng mô hình Word2Vec.
doc_words = simple_preprocess(doc)	Tiền xử lý tài liệu bằng cách sử dụng hàm simple_preprocess để chuẩn hóa văn bản (chuyển thành chữ thường, loại bỏ dấu câu, v.v.).
word_vectors = [w2v_model.wv[word] for word in doc_words if word in w2v_model.wv]	Lấy các vector của các từ trong tài liệu từ mô hình Word2Vec nếu từ đó có trong từ điển của mô hình.
return np.mean(word_vectors, axis=0) if word_vectors else np.zeros(w2v_model.vector_size)	Tính trung bình các vector từ để tạo thành vector đại diện cho tài liệu, nếu không có từ nào trong mô hình thì trả về vector không.
X_w2v = np.array([document_vector(doc) for doc in documents])	Áp dụng hàm document_vector cho tất cả các tài liệu trong documents để tạo thành ma trận đặc trưng X_w2v.
word2vec_lr_regressor = LinearRegression()	Khởi tạo mô hình hồi quy tuyến tính để dự đoán giá trị mục tiêu từ các đặc trưng đã được vector hóa.
word2vec_lr_regressor.fit(X_w2v, prices)	Huấn luyện mô hình hồi quy tuyến tính với đặc trưng X_w2v và giá trị mục tiêu prices.

```
def word2vec_lr_pricer(item):
    doc = item.test_prompt()
    doc_vector = document_vector(doc)
    return max(0, word2vec_lr_regressor.predict([doc_vector])[0])

Tester.test(word2vec_lr_pricer)
np.random.seed(42)
svr_regressor = LinearSVR()

svr_regressor.fit(X_w2v, prices)
```

Hàm/Dòng lệnh	Chức năng
def word2vec_lr_pricer(item):	Khởi tạo hàm tính giá cho sản phẩm dựa trên mô hình hồi quy tuyến tính với đặc trưng được vector hóa bằng Word2Vec.
doc = item.test_prompt()	Lấy prompt (mô tả) của sản phẩm từ đối tượng item.
doc_vector = document_vector(doc)	Tạo vector đặc trưng cho tài liệu sử dụng mô hình Word2Vec thông qua hàm document_vector.
return max(0, word2vec_lr_regressor.predict([doc_vector])[0])	Dự đoán giá trị cho sản phẩm từ mô hình word2vec_lr_regressor và đảm bảo giá trị dự đoán không nhỏ hơn 0.
Tester.test(word2vec_lr_pricer)	Chạy kiểm thử cho hàm word2vec_lr_pricer.
np.random.seed(42)	Đặt giá trị hạt giống cho bộ sinh số ngẫu nhiên, giúp tái tạo kết quả mỗi khi chạy lại mã.
svr_regressor = LinearSVR()	Khởi tạo mô hình Hồi quy tuyến tính với Support Vector Regression (SVR).
svr_regressor.fit(X_w2v, prices)	Huấn luyện mô hình SVR với đặc trưng X_w2v và giá trị mục tiêu prices.

```
def svr_pricer(item):
    np.random.seed(42)
    doc = item.test_prompt()
    doc_vector = document_vector(doc)
    return max(float(svr_regressor.predict([doc_vector])[0]), 0)
```

Hàm/Dòng lệnh	Chức năng
def svr_pricer(item):	Khởi tạo hàm tính giá cho sản phẩm dựa trên mô hình Support Vector Regression (SVR).
np.random.seed(42)	Đặt giá trị hạt giống cho bộ sinh số ngẫu nhiên, giúp tái tạo kết quả mỗi khi chạy lại mã.
doc = item.test_prompt()	Lấy prompt (mô tả) của sản phẩm từ đối tượng item.
doc_vector = document_vector(doc)	Tạo vector đặc trưng cho tài liệu sử dụng mô hình Word2Vec thông qua hàm document_vector.
return max(float(svr_regressor.predict([doc_vector])[0]), 0)	Dự đoán giá trị cho sản phẩm từ mô hình SVR và đảm bảo giá trị dự đoán không nhỏ hơn 0.
Tester.test(svr_pricer)	Chạy kiểm thử cho hàm svr_pricer.
rf_model = RandomForestRegressor(n_estimators=100,	Khởi tạo mô hình Random Forest Regressor với 100 cây quyết định, sử dụng 8 tác vụ song song và hạt giống ngẫu nhiên 42.

random_state=42, n_jobs=8)	
rf_model.fit(X_w2v, prices)	Huấn luyện mô hình Random Forest với đặc trưng X_w2v và giá trị mục tiêu prices.
def random_forest_pricer(item):	Khởi tạo hàm tính giá cho sản phẩm dựa trên mô hình Random Forest.
doc = item.test_prompt()	Lấy prompt (mô tả) của sản phẩm từ đối tượng item.
doc_vector = document_vector(doc)	Tạo vector đặc trưng cho tài liệu sử dụng mô hình Word2Vec thông qua hàm document_vector.
return max(0, rf_model.predict([doc_vector])[0])	Dự đoán giá trị cho sản phẩm từ mô hình Random Forest và đảm bảo giá trị dự đoán không nhỏ hơn 0.
Tester.test(random_forest_pricer)	Chạy kiểm thử cho hàm random_forest_pricer.